Universidad Mariano Gálvez de Guatemala
Facultad de Ingeniería en Sistemas

Nombre	Jorge Eduardo Espal Molgal	Camé	9941-24-10596

Carrera	Ingeniería en Sistemas	Cod. Carrera	9941
Asignatura	Programación III	Cod. Curso	022
Ciclo	Quinto Ciclo	Jornada	Domingos
Catedrático	Ing. MBA David Alvarez	Fecha	19/04/2026
Semestre	Primer Semestre	Sección	B

Ejercicios:

Para cada ejercicio:

1. Dibuje el árbol
2. Escriba los 3 recorridos

Ejercicio 1

Datos: 50, 30, 70, 20, 40, 60, 80

Ejercicio 2

Datos: 10, 5, 1, 7, 40, 50

Ejercicio 3

Datos: 25, 15, 10, 22, 35, 30, 40

Ejercicio 4

Datos: 8, 3, 10, 1, 6, 14, 4, 7, 13

Ejercicio 5

Datos: 100, 50, 150, 25, 75, 125, 175

Análisis

Responda:

1. ¿Qué recorrido SIEMPRE da los datos ordenados?
   _____
2. ¿Por qué ocurre eso?
   _____
3. ¿Qué pasa si los datos se insertan en orden ascendente?
   _____
4. ¿Qué forma tomaría el árbol?
   _____

Implementación en C++

Realice la siguiente implementación en C++ para los árboles BST

Estructura base

```
struct Nodo {
    int dato;
    Nodo* izquierdo;
    Nodo* derecho;
};
```

Función crear nodo

```
Nodo* crearNodo(int valor) {
    Nodo* nuevo = new Nodo();
    nuevo->dato = valor;
    nuevo->izquierdo = NULL;
    nuevo->derecho = NULL;
    return nuevo;
}
```

Inserción

```
Nodo* insertar(Nodo* raiz, int valor) {
    if (raiz == NULL) {
        return crearNodo(valor);
    }

    if (valor < raiz->dato) {
        raiz->izquierdo = insertar(raiz->izquierdo, valor);
    } else {
        raiz->derecho = insertar(raiz->derecho, valor);
    }

    return raiz;
}
```

Recorridos

Analice las siguientes funciones y utilícelas en su programa:

```
void inOrden(Nodo* raiz) {
    if (raiz != NULL) {
        inOrden(raiz->izquierdo);
        cout << raiz->dato << " ";
        inOrden(raiz->derecho);
    }
}

void preOrden(Nodo* raiz) {
    if (raiz != NULL) {
        cout << raiz->dato << " ";
        preOrden(raiz->izquierdo);
        preOrden(raiz->derecho);
    }
}

void postOrden(Nodo* raiz) {
    if (raiz != NULL) {
        postOrden(raiz->izquierdo);
        postOrden(raiz->derecho);
        cout << raiz->dato << " ";
    }
}
```

Modificar el programa para cumplir con los siguientes requisitos:

- Agregar un menú interactivo con las siguientes opciones:
  - Insertar valor en el árbol
  - Mostrar recorrido InOrden
  - Mostrar recorrido PreOrden
  - Mostrar recorrido PostOrden
  - Mostrar los 3 recorridos
  - Salir
- El programa debe seguir ejecutándose hasta que el usuario elija la opción Salir
- Cada vez que el usuario seleccione una opción de recorrido, el programa debe mostrar el resultado correspondiente
- Si el árbol está vacío, debe indicarlo con un mensaje adecuado
- Debe usarse switch
- Debe mantenerse la raíz correctamente durante toda la ejecución

Base:

```
int main() {
    Nodo* raiz = NULL;
    int valor;

    for (int i = 0; i < 6; i++) {
        cout << "Ingrese valor: ";
        cin >> valor;
        raiz = insertar(raiz, valor);
    }

    cout << "\nInOrden: ";
    inOrden(raiz);

    cout << "\nPreOrden: ";
    preOrden(raiz);

    cout << "\nPostOrden: ";
    postOrden(raiz);

    return 0;
}
```

Ejemplo esperado del menú:

```
cout << "\n--- MENU ---";
cout << "\n1. Insertar valor";
cout << "\n2. Mostrar InOrden";
cout << "\n3. Mostrar PreOrden";
cout << "\n4. Mostrar PostOrden";
cout << "\n5. Mostrar todos los recorridos";
cout << "\n6. Salir";
cout << "\nSeleccione una opcion: ";
```

REQUERIMIENTO ADICIONAL DE ENTREGA

Entrega obligatoria en GitHub

Subir su trabajo a un repositorio en GitHub.

Debe incluir:

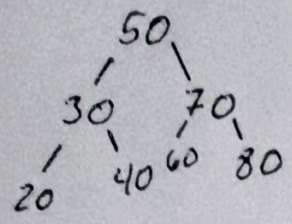
- Archivo fuente .cpp
- Capturas de pantalla del programa en ejecución
- Evidencia clara de compilación y funcionamiento

Evidencia obligatoria

Además del código en GitHub, debe adjuntar un **archivo Word o PDF** con evidencia donde se observe:

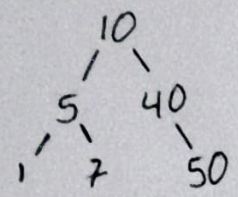
- Su **nombre completo**
- La ejecución del programa
- Inserción de datos en el árbol
- Visualización de recorridos desde el menú
- Que el programa compila y funciona correctamente

Ejercicio #1



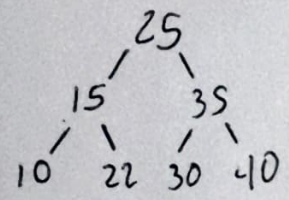
Inorden: 20, 30, 40, 50, 60, 70, 80
Preorden: 50, 30, 20, 40, 70, 60, 80
Postorden: 20, 40, 30, 60, 80, 70, 50

Ejercicio #2



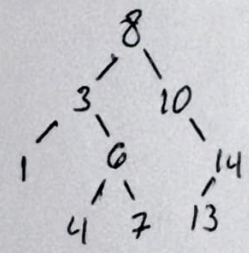
Inorden: 1, 5, 7, 10, 40, 50
Preorden: 10, 5, 1, 7, 40, 50
Postorden: 1, 7, 5, 50, 40, 10

Ejercicio #3



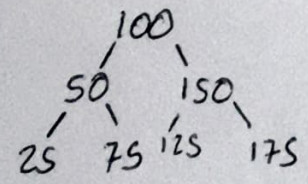
Inorden: 10, 15, 22, 25, 30, 35, 40
Preorden: 25, 15, 10, 22, 35, 30, 40
Postorden: 10, 22, 15, 30, 40, 35, 25

Ejercicio #4



Inorden: 1, 3, 4, 6, 7, 8, 10, 13, 14
Preorden: 8, 3, 1, 6, 4, 7, 10, 14, 13
Postorden: 1, 4, 7, 6, 3, 13, 14, 10, 8

Ejercicio #5



Inorden: 25, 50, 75, 100, 125, 150, 175
Preorden: 100, 50, 25, 150, 125, 175
Postorden: 25, 75, 50, 125, 175, 150, 100

Analisis:

1. El recorrido Inorden
2. Porque un arbol BST, los valores menores que dan a la izquierda y los mayores a la derecha.

3. El arbol queda desbalanciado

4. Tomaria una forma parecida a una lista inclinada hacia la derecha.